

Execution-Free Roofline Triage for GPU Kernels with Off-the-Shelf LLMs

Anonymous Author(s)

Abstract

AI coding agents increasingly propose GPU code changes, but evaluating those changes still depends on compile-run-profile loops on scarce target hardware. This paper studies a narrower software-engineering question: can off-the-shelf large language models provide an execution-free early triage signal for GPU kernels before runtime profiling is available?

We formulate the task as predicting profiler-derived DRAM-level Roofline Arithmetic Intensity (RAI) from software artifacts. For each first kernel invocation, the models predict launch dimensions, FP16/FP32/FP64 floating-point work, and DRAM bytes read and written; RAI follows from these quantities. We evaluate 254 CUDA and OpenMP kernels (95 programs, 1,016 kernel-device rows) across four NVIDIA GPUs, comparing *source-only* prompts against prompts that also include post-compilation SASS.

Across the completed evaluation subset, source-only reasoning is often sufficient for nonzero FP32/FP64 triage, but not for FP16. Adding SASS produces the strongest gains for the frontier models, especially for FP16 and nonzero bandwidth-bound versus compute-bound decisions. When rows are ranked by predicted RAI relative to the GPU balance point, the top 10% of frontier-model *source+SASS* predictions recover all shared-subset FP16 compute-bound rows, while source-only FP32 rankings already recover 82–96%. Simple deterministic and lightweight learned static baselines recover part of the same signal, underscoring the benchmark value beyond a single model family. The strongest frontier LLMs remain best on *source+SASS* FP16/FP32 triage, but lightweight learned baselines are competitive in some source-only FP16 and *source+SASS* FP64 regimes while remaining poor numeric RAI regressors. We also package the benchmark, prompts, responses, and plotting artifacts for follow-on work; public URLs are omitted in this anonymous version. Our claim is therefore limited and practical: off-the-shelf LLMs can provide an execution-free triage signal for GPU kernels, but they do not replace profiling.

CCS Concepts

• **Software and its engineering** → **Software performance**; *Software prototyping*; • **Mathematics of computing** → *Performance analysis*.

Keywords

software performance, GPUs, Roofline, arithmetic intensity, large language models, static performance reasoning

ACM Reference Format:

Anonymous Author(s). 2027. Execution-Free Roofline Triage for GPU Kernels with Off-the-Shelf LLMs. In *International Conference on Software Engineering (ICSE)*, Submission Version. ACM, New York, NY, USA, 14 pages.

ICSE Submission,
2027.

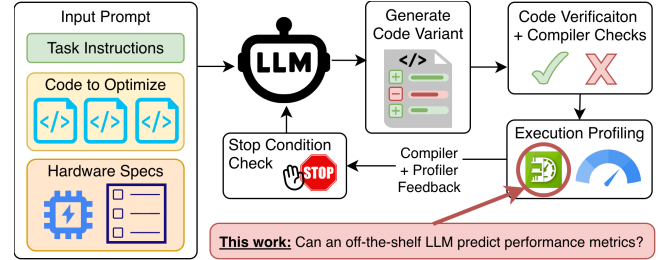


Figure 1: Profiler-in-the-loop GPU optimization is effective, but it assumes repeated access to the target accelerator. This paper studies whether LLMs can provide an execution-free triage signal earlier in that loop, before runtime profiling is available.

1 Introduction

GPU performance engineering is still fundamentally measurement-driven. Developers usually optimize kernels by compiling candidate variants, running them on the target accelerator, profiling hardware counters, and then deciding whether to focus on computation, memory traffic, or both. That workflow is already costly for humans, and it becomes even more expensive in agentic software-engineering loops where an LLM proposes many candidate edits and expects profiler feedback after each one [5, 7, 13, 14, 18, 25].

In practice, target-GPU access is often the bottleneck. Accelerators may be rationed on a shared cluster, unavailable in early development, blocked by missing system dependencies, or simply too expensive to occupy for speculative triage. The problem is therefore not “can LLMs replace profilers?”; it is whether they can provide a smaller, earlier, and still interpretable signal that helps developers and tools decide where scarce profiling effort should be spent first.

We focus on one such signal: Roofline Arithmetic Intensity (RAI), the ratio of floating-point work to DRAM traffic in the Roofline model [23]. RAI is attractive for software engineering because it is both profiler-relevant and interpretable. It does not directly predict runtime, but it does give a developer or tool builder an early indication of whether a kernel is likely to sit on the memory side or the compute side of a target GPU’s Roofline balance point. That makes it a plausible triage target for LLM-based development tools that must reason from software artifacts before execution.

This paper studies whether off-the-shelf LLMs can estimate profiler-derived RAI from source code, build context, command-line inputs, and optional post-compilation artifacts such as SASS. Our setting is execution-free at prediction time: the models do not execute or profile the target kernel when producing a prediction. Ground truth, however, is profiler-derived and comes from an offline benchmark-building process. We therefore ask a deliberately

narrower question than general performance prediction: *Can off-the-shelf LLMs provide an execution-free early triage signal for GPU kernels by estimating DRAM-level RAI from software artifacts?*

To answer that question, we build on 95 HeCBench CUDA and OpenMP programs. They expose 254 first-invocation GPU kernels and 1,016 kernel-device rows across an RTX 3080, A10, A100, and H100. We compare *source-only* prompts against *source+SASS* prompts for three off-the-shelf models: GPT-5.4, Opus 4.6, and GPT OSS. The released evaluation subset contains 5,766 completed model queries, which yield 17,298 precision-specific FP16/FP32/FP64 RAI predictions. This lets us study not only numerical RAI error, but also zero-versus-nonzero detection, nonzero bandwidth-bound versus compute-bound triage, prompt-evidence effects, cost/latency tradeoffs, and recurring failure patterns.

Three findings matter most. First, source-only reasoning is often good enough for nonzero FP32/FP64 triage, but it collapses for FP16. Second, SASS helps substantially when compiler-realized behavior matters, especially for FP16 and for frontier-model triage decisions. On the 732 kernel-device rows shared across all paper configurations, *source+SASS* improves nonzero FP16 balanced triage accuracy from the trivial 0.50 baseline to 0.870 for GPT-5.4 and 0.984 for Opus 4.6. Third, the signal is strong enough to prioritize scarce profiling effort: at a 10% profiling budget, *source+SASS* GPT-5.4 and Opus 4.6 recover all shared-subset FP16 compute-bound rows, while source-only FP32 rankings already recover 82% and 96% of the true compute-bound rows. The benchmark is also not “LLMs versus nothing”: context-only, deterministic, and lightweight learned static baselines recover substantial triage signal and even exceed one-shot prompting in a few source-only FP16 and *source+SASS* FP64 settings, but they remain poor numeric RAI regressors and still trail the frontier models on *source+SASS* FP16/FP32 triage. The cheapest model is still a useful low-cost source-only point, but it is less complete and makes far weaker use of SASS than the frontier models.

The paper’s contribution is therefore squarely about software engineering. We study how execution-relevant software artifacts can support AI-assisted performance reasoning before execution. We package that task as a benchmark and empirical study, show where off-the-shelf prompting is already useful, and identify where it remains unreliable.

More concretely, this paper makes four contributions:

- We formulate execution-free RAI prediction as a software-engineering task for early GPU-kernel triage. Inputs are software artifacts, and ground truth is profiler-derived DRAM-level RAI.
- We report an empirical study across 254 CUDA/OpenMP kernels and four NVIDIA GPUs, with explicit task splits for zero/nonzero detection, nonzero RAI regression, nonzero BB/CB triage, and fixed-budget compute-bound retrieval.
- We quantify the effect of post-compilation artifacts against context-only, deterministic, and lightweight learned static baselines. SASS materially helps the frontier models, but different static methods win in different regimes.
- We package the benchmark, prompts, responses, and plotting artifacts for follow-on work, while documenting the method’s

limits so future work can compare against stronger static baselines and better-validated failure analyses.

The remainder of the paper is organized as follows. Section 2 positions the paper relative to Roofline analysis, execution-free GPU performance prediction, and LLM-based optimization. Sections 3 and 4 define the prediction task and study design. Section 5 answers the research questions. Sections 6 and 7 translate the findings into implications and limitations for ICSE readers, and Section 8 concludes.

2 Background and Related Work

This paper sits at the intersection of Roofline-guided GPU performance engineering, execution-free performance prediction, and LLM-based software engineering. The relevant prior work is broad, but the distinctions that matter for this paper are fairly crisp.

Roofline-guided GPU triage. The Roofline model gives a compact way to reason about whether a kernel is limited primarily by computation or by data movement [23]. Its horizontal axis, arithmetic intensity, is not a full performance model; it is an interpretable ratio that helps developers decide which optimization direction is worth pursuing first. That interpretability is why we use RAI as a triage target rather than a more opaque prediction objective. The paper’s contribution is therefore not another Roofline model, but an empirical study of whether LLMs can estimate a Roofline-relevant signal from software artifacts before target execution.

Execution-free GPU performance prediction. Long before LLMs, GPU researchers studied analytical and learned predictors that estimate performance without executing the target kernel. Their inputs include static program structure, microarchitectural assumptions, PTX/SASS features, and compiler-derived summaries [1, 2, 4, 6, 8, 9, 21]. These works establish that execution-free prediction is valuable, but they generally target runtime, throughput, power, or energy rather than an interpretable profiler-relevant triage signal. They also usually rely on task-specific modeling pipelines rather than off-the-shelf prompting. Our question is narrower and more software-engineering-oriented: what can a developer tool obtain from existing code/build artifacts using an unmodified LLM?

LLMs for GPU optimization and code reasoning. Recent work has shown that LLMs can generate or optimize accelerator code, especially when compilation, execution, testing, or profiling is available inside the loop [5, 7, 13, 18, 19, 25]. That literature is directly relevant to ICSE because it frames GPU tuning as an AI-assisted software-development activity. Our paper complements those systems rather than competing with them. We do not optimize kernels directly; we study whether an LLM can provide an earlier triage signal when the profiler loop itself is unavailable or too costly.

LLMs for performance prediction. The closest neighboring work asks LLMs to predict GPU metrics directly. LLMPerf fine-tunes models for GPU performance estimation [17]; Omniwise studies fine-tuned prediction of several GPU metrics, including arithmetic intensity [22]; and prior work by Bolet *et al.* studies coarse BB/CB classification rather than quantitative RAI prediction [3]. Our contribution differs in three ways. First, we study the zero-training regime with off-the-shelf models rather than fine-tuned predictors.

Second, we target a quantitative, decomposed signal by asking models to predict launch dimensions, per-precision FLOP counts, and DRAM bytes, from which RAI follows. Third, we explicitly compare *source-only* reasoning with *source+SASS* reasoning to isolate the gap between source-visible semantics and compiler-realized behavior.

Static reasoning with code LLMs. Our task also relates to a broader line of work showing that code LLMs can reason about latent program behavior, but only imperfectly [10, 15, 16, 20, 24]. Estimating RAI requires more than recognizing syntax: the model must propagate launch parameters, recover thread-level work, account for mixed precision, and reason about DRAM-visible traffic instead of source-level loads and stores. That makes RAI prediction a useful probe of execution-relevant code reasoning in a form that is directly actionable for software engineering.

Gap addressed by this paper. Taken together, prior work leaves an open gap. We know that static models can predict GPU behavior without execution, that LLM optimizers benefit from profiler feedback, and that fine-tuned LLM predictors can estimate GPU metrics. What remains underexplored is whether *off-the-shelf* LLMs can provide a useful execution-free triage signal from ordinary software artifacts alone. This paper addresses that gap by studying quantitative RAI prediction, by separating source-visible from post-compilation evidence, and by positioning the released benchmark as infrastructure for future static performance-reasoning work in software engineering.

3 Task Formulation and Methodology

3.1 Prediction Task

Our target is the DRAM-level Roofline Arithmetic Intensity (RAI) of a GPU kernel’s *first invocation*. At prediction time, the LLM does not execute the program, collect profiler counters, or observe runtime traces. Instead, it receives software artifacts and must infer the quantities from which RAI is derived.

We do not ask the model to emit RAI directly. For each kernel-device pair, the model predicts seven values:

- (1) grid size,
- (2) block size,
- (3) FP16 operation count,
- (4) FP32 operation count,
- (5) FP64 operation count,
- (6) DRAM bytes read, and
- (7) DRAM bytes written.

We then derive RAI separately for each precision:

$$RAI_p = \frac{FLOP_p}{BytesRead + BytesWritten}, \quad p \in \{FP16, FP32, FP64\}.$$

This decomposition makes failures easier to interpret. An incorrect launch configuration propagates into total work; an incorrect FLOP count indicates semantic or compiler-lowering errors; and an incorrect byte estimate indicates difficulty reasoning about DRAM-visible traffic.

We intentionally keep separate FP16/FP32/FP64 RAI values rather than collapsing mixed-precision kernels into a single scalar. This matches the per-precision balance points used later for triage and makes compiler-injected FP16 work visible. It also means that

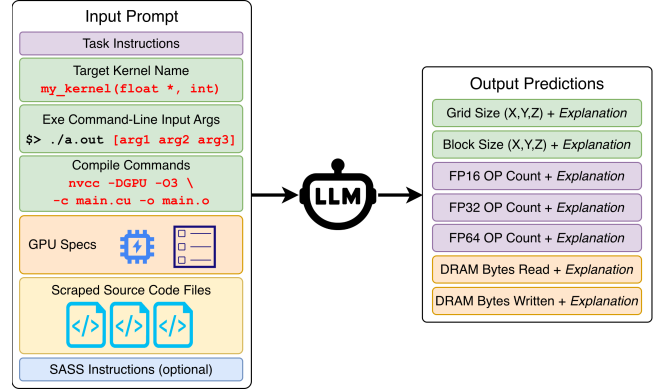


Figure 2: Per-kernel prompting workflow. Each query includes the target kernel name, source files, compile commands, runtime arguments, and GPU specifications. The *source+SASS* setting adds target-kernel SASS sections and relevant helper routines so we can measure the value of post-compilation evidence.

a kernel may have zero FP64 RAI but nonzero FP32 RAI, which we treat as a feature of the task rather than a modeling mistake.

3.2 Input Evidence

Figure 2 summarizes the evidence we provide to the model. Every prompt contains the kernel identifier (mangled and demangled), relevant source files, compile commands, runtime arguments, and target-GPU specifications. The *source+SASS* configuration augments the same prompt with SASS sections for the target kernel and helper routines that may execute on its path.

The distinction between *source-only* and *source+SASS* is important. *source-only* corresponds to early development, code review, or agent planning when only pre-compilation artifacts are available. *source+SASS* still avoids target execution and profiling at prediction time, but it assumes target-specific compilation is possible, which is a materially different stage of the workflow. We therefore treat *source+SASS* as *execution-free* rather than broadly “hardware-free.”

The model returns a structured tool-call response containing the seven predicted quantities plus short explanations. Those explanations are useful for sanity checking and for the later exploratory failure analysis, but they are not used to compute the reported metrics.

3.3 Profiler-Derived Ground Truth

Ground truth comes from an offline benchmark-building process, not from the prediction loop itself. We profile each benchmark with NVIDIA Nsight Compute and retain the first invocation of the target kernel in each run. We repeat the executable three times and average the retained first-invocation counters to reduce profiling noise.

The exact DRAM-traffic counters are `dram__bytes_read.sum` and `dram__bytes_write.sum`. Floating-point counts are reconstructed from executed-operation counters:

- FP32:


```

smp_sass_thread_inst_executed_op_fadd_pred_on.sum.per_
cycle_elapsed, smp_sass_thread_inst_executed_op_fmnl_pred_on.
sum.per_cycle_elapsed,
derived_smp_sass_thread_inst_executed_op_ffma_pred_on.x2.

```

- FP64:

```

smp_sass_thread_inst_executed_op_dadd_pred_on.sum.per_
cycle_elapsed, smp_sass_thread_inst_executed_op_dmul_pred_on.
sum.per_cycle_elapsed,
derived_smp_sass_thread_inst_executed_op_dfma_pred_on.x2.

```

- FP16:

```

smp_sass_thread_inst_executed_op_hadd_pred_on.sum.per_
cycle_elapsed, smp_sass_thread_inst_executed_op_hmul_pred_on.
sum.per_cycle_elapsed,
derived_smp_sass_thread_inst_executed_op_hfma_pred_on.x4.

```

We convert those per-cycle rates to total operation counts using `smp_cycles_elapsed.avg.per_second` and `gpu_time_duration.sum`.

These choices matter for scope. Because bytes are counted at the DRAM interface, the task asks the model to predict *DRAM-visible* traffic rather than raw source-level memory operations. Writes that remain in the cache hierarchy until later are not immediately visible in the denominator. Likewise, compiler-injected helper routines for division or transcendental functions contribute to the measured FLOP counts even when they are not obvious in source. Those are precisely the cases where *source+SASS* may add value.

3.4 Why First Invocation and DRAM-Level RAI

We make two deliberate simplifications. First, we evaluate only the first invocation of each kernel, because repeated invocations introduce temporal cache reuse and cross-kernel interference that would substantially widen the reasoning space. Second, we focus on DRAM-level RAI rather than a cache-aware Roofline. This keeps the prediction target tied to one interpretable denominator while still exposing realistic sources of error such as L2 write-back behavior.

These simplifications narrow the claim. The paper studies execution-free early triage for a first kernel invocation at the DRAM level; it does not claim to predict steady-state runtime behavior, cache-resident arithmetic intensity, or end-to-end application performance.

4 Study Design

4.1 Benchmark Corpus

We draw workloads from the HeCBench suite [11], focusing on CUDA and OpenMP target-offload programs because they are both common GPU programming models and represent distinct source/build ecosystems [12]. The final corpus contains 95 benchmark programs: 56 CUDA programs and 39 OpenMP programs. Those programs expose 254 first-invocation GPU kernels: 155 CUDA kernels and 99 OpenMP kernels.

Profiling the corpus on four NVIDIA GPUs yields 1,016 kernel-device rows. Each row induces three precision-specific RAI labels (FP16/FP32/FP64), so the profiler-derived ground-truth space contains 3,048 precision-specific evaluation points before any model-query filtering.

We profile on four GPUs chosen to span different memory and compute regimes: RTX 3080, A10, A100, and H100. Table 1 lists the hardware characteristics and the resulting DRAM-level balance points used for BB/CB classification. The paper’s architectural question is therefore not whether models generalize to all accelerators;

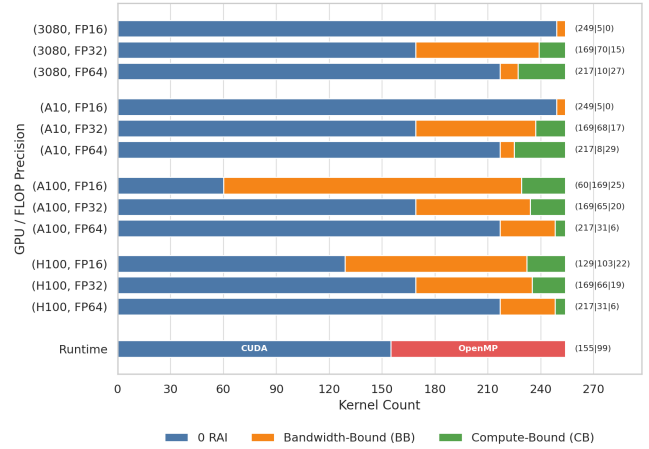


Figure 3: Ground-truth RAI class distribution by GPU and precision. Zero-valued RAI cases dominate every precision, FP16 nonzero cases are concentrated on A100/H100, and the nonzero class mix varies by device. These imbalances motivate separate evaluation of zero detection, nonzero regression, and nonzero BB/CB triage.

it is whether the same prompting strategy remains useful across noticeably different NVIDIA deployment targets.

Figure 3 highlights the resulting class imbalance. Across the full 1,016 kernel-device rows, FP16 has 687 zero cases, FP32 has 676, and FP64 has 868. FP16 is especially architecture-sensitive because the A100/H100 binaries contain many more nonzero FP16 measurements than the 3080/A10. This is precisely why we separate the evaluation tasks instead of reporting a single aggregate error number.

4.2 Models, Prompts, and Sample Accounting

We study three off-the-shelf LLMs: GPT-5.4, Opus 4.6, and the cheaper GPT OSS baseline. All reported comparisons use the two released prompt configurations that differ only by the presence or absence of SASS: *source-only* and *source+SASS*. We do not fine-tune the models, train task-specific heads, or expose runtime feedback inside the prediction loop. The committed run script invokes one trial per kernel-device pair for the model identifiers `openai/gpt-5.4`, `anthropic/claude-opus-4.6`, and `openai/gpt-oss-120b`; the released OpenRouter client configuration defaults to temperature 0.2 and top-*p* 0.1.

The expensive frontier models were queried once per kernel-device pair in the released evaluation subset. That is a methodological limitation of the artifact, so we report coverage explicitly rather than hiding it inside aggregate metrics. The completed evaluation subset contains:

- 3,048 *source+SASS* and 3,048 *source-only* predictions for Opus 4.6,
- 3,048 *source+SASS* and 3,045 *source-only* predictions for GPT-5.4, and
- 2,535 *source+SASS* and 2,574 *source-only* predictions for GPT OSS.

Table 1: Profiled GPUs and DRAM-level balance points used for BB/CB classification.

	RTX 3080	A10	A100	H100
Base architecture	Ampere (GA102)	Ampere (GA102)	Ampere (GA100)	Hopper (GH100)
Compute capability	8.6	8.6	8.0	9.0
Memory bandwidth (GB/s)	760	600	1,555	3,360
L2 cache (MB)	5	6	40	50
FP16 peak (TFLOP/s)	30.55	15.62	77.97	133.82
FP32 peak (TFLOP/s)	30.55	15.62	19.49	66.91
FP64 peak (TFLOP/s)	0.477	0.244	9.75	33.45
FP16 balance point (FLOP/byte)	40.20	26.03	50.14	39.83
FP32 balance point (FLOP/byte)	40.20	26.03	12.53	19.91
FP64 balance point (FLOP/byte)	0.63	0.41	6.27	9.96

For fair cross-model comparisons, we use the 732 kernel-device rows completed by all three models in both prompt settings. Within-model source-versus-SASS comparisons are paired on the subset completed by both settings for that model.

4.3 Evaluation Tasks and Metrics

The evaluation intentionally separates four related but distinct tasks:

Task A: zero vs. nonzero detection. We first ask whether a model can distinguish kernels whose per-precision RAI is zero from kernels with nonzero RAI. Because the obvious trivial baseline is to predict zero everywhere, we report balanced accuracy rather than raw accuracy. Only an exact zero numeric prediction counts as “zero”; any positive or non-finite prediction is conservatively treated as “nonzero.”

Task B: nonzero RAI regression. For rows whose ground-truth RAI is nonzero, we report median absolute percent error. The distributions are heavy-tailed, so we emphasize medians and the supporting boxplots rather than means. As a numeric lower bound, we also compare against a trivial global-median-RAI heuristic, which produces approximately 98.5%, 99.9%, and 99.6% median APE for FP16, FP32, and FP64 respectively on the shared subset.

Task C: nonzero BB/CB triage. For nonzero cases, we compare the expected and predicted side of the GPU-specific Roofline balance point. We report balanced accuracy because the class ratios are uneven. Appendix Table 7 complements that metric with compute-bound precision/recall/F1 so reviewers can see whether a method reaches high recall by over-predicting the compute side. If a model predicts zero for a truly nonzero kernel, we count that as a memory-side failure and collapse it onto the bandwidth-bound side for this binary triage metric. This is conservative, but appropriate: a zero prediction still fails to identify compute-side behavior.

Task D: three-way triage. Finally, we retain the full {zero, BB, CB} label space to show whether a model preserves the higher-level triage outcome that a developer or agent would see. We report multiclass balanced accuracy (macro recall). As in Task C, non-finite predictions are conservatively counted as memory-side failures rather than as a separate abstain class.

We include two trivial class baselines for context. Always predicting zero yields 0.50 balanced accuracy on Task A and 0.33 balanced

accuracy on Task D. Always predicting the majority nonzero class (bandwidth-bound in all three precisions here) yields 0.50 balanced accuracy on Task C.

We also add three simple non-LLM reference baselines. The *context median* baseline uses grouped cross-validation and predicts the training-fold median RAI for each runtime×GPU pair, with runtime-only, GPU-only, and global fallbacks when a pair is unseen. This is a deliberately naive context-only baseline that uses no code or disassembly evidence. We also add two deterministic non-LLM reference heuristics derived from the released artifact. The *source lexical* heuristic uses only kernel-linked source files: if no precision-relevant floating-point type or math-library token appears, it predicts zero; otherwise it estimates RAI as approximate arithmetic-token count divided by four times approximate memory-reference count. The *SASS mnemonic* heuristic maps each GPU to the released architecture-specific static instruction mix, counts precision-relevant floating-point arithmetic mnemonics, counts global-memory mnemonics, and converts those counts into an approximate static RAI using 2/4/8-byte denominators for FP16/FP32/FP64. These baselines are intentionally coarse. They ignore loop trip counts, control-flow frequencies, vector widths, and dynamic access sizes, so we use them only as deterministic reference points rather than as optimized competitors.

We additionally add lightweight learned static baselines from two tree families: random forests and extra trees. For each prompt setting and family, we run a five-fold GroupKFold evaluation grouped by program. Stage 1 predicts zero versus nonzero RAI; Stage 2 regresses $\log(1 + RAI)$ on the nonzero training rows and emits zero whenever Stage 1 predicts zero. The *source-only* setting uses only source-derived lexical counts plus runtime and GPU identifiers; the *source+SASS* setting adds architecture-specific static instruction-mix counts derived from the released post-compilation artifact. The main appendix table reports the seed-0 runs, and Appendix Table 10 summarizes five-seed sensitivity for the learned rows. Because these baselines are trained and evaluated only within the released corpus, we use them as lightweight static references rather than as production-quality predictors.

4.4 Exploratory Failure Analysis

The released artifact also contains an auxiliary feature-voting pipeline. It uses inexpensive LLMs to label source patterns such as division, special math, reusable subexpressions, and loop-invariant

floating-point work. We retain that analysis because it helps characterize where prediction errors cluster, but we explicitly treat it as exploratory. The feature labels are not human-validated in the released data, so the paper uses them to describe repeated failure patterns rather than to make causal claims.

5 Results

Unless stated otherwise, the main cross-model comparisons in this section use the 732 kernel-device rows completed by all three models in both prompt settings. That avoids coverage bias from the incomplete GPT OSS runs while preserving paired source-versus-SASS comparisons.

5.1 RQ1: How Accurate Is Execution-Free RAI Triage Across the Corpus?

Answer. Off-the-shelf LLMs already provide useful execution-free triage for many nonzero FP32/FP64 cases, but source-only FP16 prediction is effectively at the trivial baseline.

Table 2 summarizes the main released comparison subset. Two points are immediate. First, all three source-only FP16 configurations collapse to the trivial regime: every model has 100% median nonzero APE, and both GPT-5.4 and Opus 4.6 sit at 0.50 balanced accuracy for nonzero BB/CB triage. This is indistinguishable from always predicting the memory side. Second, source-only FP32/FP64 results are already materially better for the frontier models. On the shared rows, Opus 4.6 reaches median nonzero APE of 43.9% for FP32 and 52.0% for FP64, with balanced triage accuracy of 0.94 and 0.76 respectively. GPT-5.4 is weaker but still usable for triage at 59.2%/74.5% median nonzero APE and 0.89/0.67 balanced accuracy. Even the weakest source-only FP32/FP64 configurations substantially outperform the trivial global-median-RAI heuristic, whose median APE is effectively 100% on the shared subset.

The zero-aware tasks tell the same story. Source-only FP16 zero-versus-nonzero detection stays near the trivial 0.50 baseline for all three models (0.52 for GPT-5.4, 0.51 for GPT OSS, and 0.52 for Opus 4.6). By contrast, source-only FP32/FP64 zero detection is already strong, reaching 0.86/0.89 balanced accuracy for GPT-5.4 and 0.85/0.88 for Opus 4.6. The stricter three-way {zero, BB, CB} view is lower, as expected, but it preserves the same ranking: source-only FP16 remains near the 0.33 trivial baseline, while source-only FP32/FP64 rise to 0.82/0.68 for GPT-5.4 and 0.85/0.73 for Opus 4.6. Appendix Table 6 makes the shared-subset skew explicit: FP16/FP32/FP64 contain 513/496/623 zero rows out of 732, leaving only 34/50/53 compute-bound rows. That is why the balanced task splits matter most in FP16 and FP64, where raw accuracy would largely reflect the zero majority rather than the usefulness of the triage signal.

Those gains are not explained by a simple lexical rule, but they are also not the only source-level baseline worth taking seriously. Appendix Table 8 adds both deterministic and lightweight learned static references. The grouped runtime×GPU median baseline already captures some of the zero-heavy FP16 structure (0.790 Task A and 0.535 Task D), but it collapses to the trivial 0.50/0.333 regime on FP32/FP64, so simple execution-context priors do not explain the frontier source-only results. The deterministic source heuristic stays near trivial: it reaches Task D balanced accuracy of only 0.444

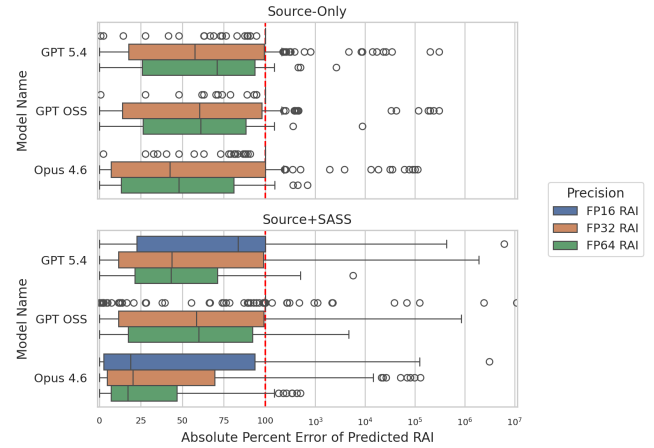


Figure 4: Absolute percent error on nonzero RAI cases. Source-only FP16 collapses at the 100% ceiling for all models, while source+SASS sharply improves FP16 for the frontier models and also tightens many FP32/FP64 distributions. Lower is better.

for FP32 and 0.531 for FP64, with median nonzero APE near 100% on both precisions. The grouped source learned baselines show that source-only FP16 collapse is not inevitable for a supervised static method: the source random-forest and extra-trees rows reach 0.688–0.699 Task C and 0.691–0.697 Task D in FP16, far above the one-shot source-only LLM results. They are still poor numeric regressors, however, with 141.6–217.4% FP16 median nonzero APE and 100% median nonzero APE on FP32/FP64. That is better read as corpus-level triage learning than as accurate numeric RAI prediction. Appendix Table 10 shows that the learned-baseline ranking is stable across five seeds: Task C standard deviations stay below 0.034. On FP32/FP64 binary triage, the frontier source-only LLMs remain at least competitive and often stronger.

Figure 4 shows the same pattern distributionally. The nonzero error distributions are extremely heavy-tailed, which is why we emphasize medians, balanced accuracy, and explicit task splits. The figure also makes it clear that GPT OSS should be read as a low-cost operating point rather than a full substitute for the frontier models: its source-only FP32/FP64 behavior is not catastrophic, but its FP16 behavior remains pinned at the ceiling and its completion coverage is materially lower.

The GPU and runtime effects are real, but they are uneven rather than universal. The appendix GPU-stratified and runtime-stratified plots show lower FP32/FP64 absolute errors on A100/H100 than on 3080/A10, consistent with the original observation that smaller-cache GPUs make DRAM-visible traffic harder to infer. The runtime split also remains precision-sensitive: CUDA tends to be easier for FP16/FP32 triage, while FP64 behavior is more mixed across CUDA and OpenMP.

Practical implication. If only source-level evidence is available, off-the-shelf prompting is already a plausible early triage tool for many FP32/FP64 cases, but not for FP16. Any tool that uses source-only LLM triage should therefore surface its FP16 uncertainty explicitly rather than treating all precisions symmetrically.

Table 2: Shared-subset summary on the 732 kernel-device rows completed by all three models in both prompt settings. “MedAPE” is the median absolute percent error on nonzero cases. “BalAcc” is balanced accuracy on nonzero BB/CB triage. Median cost and time are per-query values on the same shared subset.

Model	Evidence	Rows	FP16 MedAPE	FP16 BalAcc	FP32 MedAPE	FP32 BalAcc	FP64 MedAPE	FP64 BalAcc	Median \$	Median s
GPT-5.4	<i>source-only</i>	732	100.0	0.500	59.2	0.889	74.5	0.670	0.0197	11.3
GPT-5.4	<i>source+SASS</i>	732	84.2	0.870	34.9	0.895	35.4	0.737	0.0315	12.3
GPT OSS	<i>source-only</i>	732	100.0	0.500	58.9	0.875	62.4	0.690	0.0006	33.2
GPT OSS	<i>source+SASS</i>	732	100.0	0.583	61.4	0.865	58.4	0.652	0.0008	32.5
Opus 4.6	<i>source-only</i>	732	100.0	0.500	43.9	0.940	52.0	0.755	0.0679	22.5
Opus 4.6	<i>source+SASS</i>	732	20.4	0.984	18.3	0.967	15.3	0.765	0.1066	29.8

5.2 RQ2: Does Post-Compilation Information Improve Triage?

Answer. Yes, decisively for FP16 and materially for the frontier models; the effect is smaller and less consistent for FP32/FP64, and much weaker for GPT OSS.

The source-versus-SASS differences are best read as paired deltas on nonzero rows. For Opus 4.6, *source+SASS* lowers median nonzero FP16 APE by 78.9 points and raises nonzero balanced triage accuracy from 0.50 to 0.984. For GPT-5.4, the corresponding FP16 triage lift is from 0.50 to 0.870, with 68.9% of paired nonzero FP16 rows improving. These gains are exactly what we would expect when source-visible semantics hide compiler-injected FP16 work.

The FP32/FP64 story is more nuanced. For Opus 4.6, *source+SASS* still improves median nonzero APE from 43.9% to 18.3% in FP32 and from 52.0% to 15.3% in FP64, but the balanced BB/CB lift is much smaller because source-only triage was already strong. GPT-5.4 also improves numerically in FP32/FP64, but less dramatically. GPT OSS shows almost no meaningful FP16 response to SASS and only mixed FP32/FP64 changes.

The paired evidence is stronger than the aggregate medians alone suggest. On paired nonzero rows, exact sign tests favor *source+SASS* over *source-only* for every frontier-model precision setting: for GPT-5.4, $p = 4.7 \times 10^{-12}$, 2.5×10^{-3} , and 1.1×10^{-3} across FP16/FP32/FP64; for Opus 4.6, $p = 9.7 \times 10^{-15}$, 2.5×10^{-4} , and 1.8×10^{-4} . For the largest FP16 effects, paired bootstrap 95% confidence intervals on the median absolute-percent-error reduction are [0.8, 26.9] for GPT-5.4 and [54.7, 87.5] for Opus 4.6. The FP64 intervals remain positive for Opus 4.6 ([0.01, 30.5]) and touch zero for GPT-5.4 ([0.0, 23.8]), while FP32 intervals are narrower and often pinned near zero despite significant sign tests. Exact McNemar tests on Task C likewise confirm the FP16 classification lift for GPT-5.4 and Opus 4.6 ($p = 3.8 \times 10^{-3}$ and 7.1×10^{-6}), while FP32/FP64 paired classification gains are smaller and not consistently significant because source-only triage is already strong there.

Appendix Table 8 also shows that post-compilation artifacts carry real signal even without an LLM. The SASS mnemonic baseline already lifts zero/nonzero and three-way triage, and the learned SASS tree baselines reach 0.737–0.739 Task C balanced accuracy in FP32 and 0.744–0.824 in FP64. On SASS-backed FP64 binary triage the random-forest baseline exceeds the frontier LLMs, and the mnemonic baseline is even stronger in the budgeted retrieval setting, suggesting that compact post-compilation counts already capture much of that regime. But those same static baselines remain

weak on FP16 (best Task C 0.591) and poor numeric regressors in FP16/FP32, so they do not eliminate the need for richer reasoning where compiler-realized work and memory effects are harder to reduce to counts alone. The frontier LLMs remain decisively strongest on *source+SASS* FP16 and still lead the main FP32 triage comparison.

This is the central methodological lesson of the paper. Post-compilation evidence matters most when the source no longer exposes the computation that the profiler ultimately counts. SASS is not universally required, but it is often the difference between trivial and useful FP16 triage.

Practical implication. There are really two workflows here. *source-only* is the right deployment point for pre-compilation review and design-time triage. *source+SASS* is the right deployment point when compilation for the target architecture is possible but execution or profiling is still unavailable. Tools should not collapse those workflows into a single “hardware-free” claim.

5.3 RQ3: Where Do Frontier LLMs Beat Cheap Static Alternatives, and Where Do They Not?

Answer. Frontier LLMs dominate the main FP32 triage regimes and *source+SASS* FP16, but cheap static methods already match or exceed point estimates in *source-only* FP16 and *source+SASS* FP64.

Table 3 makes the regime boundaries explicit by comparing the best off-the-shelf LLM and the best static reference separately within each prompt/precision cell. The strongest paired LLM wins are *source-only* FP32 (0.940 versus 0.815, 44 versus 3 discordant wins, $p < 0.001$), *source+SASS* FP16 (0.984 versus 0.591, 35 versus 5, $p < 0.001$), and *source+SASS* FP32 (0.967 versus 0.739, 46 versus 2, $p < 0.001$). These are exactly the regimes where richer reasoning about lowered arithmetic and DRAM-visible traffic appears to matter most.

The static side is still real. In *source-only* FP16, the best grouped source baseline reaches 0.699 Task C balanced accuracy while every source-only LLM remains at the 0.50 trivial baseline. In *source+SASS* FP64, the best learned SASS baseline reaches 0.824 versus 0.765 for the best LLM. Those are point-estimate advantages rather than statistically decisive paired wins on the shared subset (exact $p = 0.210$ and $p = 0.238$), which is unsurprising given that FP16 and FP64 contain only 34 and 53 compute-bound rows respectively. Still, together with the budget-sweep table, they show that some regimes are already recoverable from cheap static features after lowering.

Numeric RAI estimation remains the sharper separator. Even where static baselines are competitive for triage, their nonzero median APE remains much larger than the strongest frontier LLM

Table 3: Best-off-the-shelf LLM versus best static reference for Task C nonzero BB/CB triage on the shared subset. Each cell reports balanced accuracy and nonzero MedAPE for the strongest method in that evidence family, selected separately within each prompt/precision regime. The discordant-pair columns report exact paired wins for Task C correctness on the same rows.

Evidence	Precision	Best LLM	Task C / MedAPE	Best static	Task C / MedAPE	Discordant wins LLM/static (p)
<i>source-only</i>	FP16	GPT-5.4	0.500 / 100.0	Source learned ET	0.699 / 217.4	8/15 (0.210)
<i>source-only</i>	FP32	Opus 4.6	0.940 / 43.9	Source learned RF	0.815 / 100.0	44/3 (<0.001)
<i>source-only</i>	FP64	Opus 4.6	0.755 / 52.0	Source lexical	0.683 / 99.5	21/13 (0.229)
<i>source+SASS</i>	FP16	Opus 4.6	0.984 / 20.4	SASS learned ET	0.591 / 178.6	35/5 (<0.001)
<i>source+SASS</i>	FP32	Opus 4.6	0.967 / 18.3	SASS learned ET	0.739 / 100.0	46/2 (<0.001)
<i>source+SASS</i>	FP64	Opus 4.6	0.765 / 15.3	SASS learned RF	0.824 / 67.1	6/12 (0.238)

Table 4: Compute-bound recall when profiling only the top 10% of rows ranked by predicted RAI relative to the GPU balance point on the 732-row shared subset. Each precision budgets 74 of 732 rows. Random ranking would recover 10% by construction.

Model	Evidence	FP16 R@10%	FP32 R@10%	FP64 R@10%
GPT-5.4	<i>source-only</i>	0.000	0.820	0.566
GPT-5.4	<i>source+SASS</i>	1.000	0.820	0.585
GPT OSS	<i>source-only</i>	0.000	0.820	0.547
GPT OSS	<i>source+SASS</i>	0.176	0.820	0.566
Opus 4.6	<i>source-only</i>	0.000	0.960	0.585
Opus 4.6	<i>source+SASS</i>	1.000	0.940	0.566

results in the corresponding cells: 217% versus 100% for *source-only* FP16, 100% versus 43.9% for *source-only* FP32, and 67.1% versus 15.3% for *source+SASS* FP64. The static baselines therefore look better as queueing or routing components than as direct numeric RAI estimators.

Practical implication. The right takeaway is not “LLMs beat static methods everywhere” or the reverse. A practical tool should route by regime: cheap static filters are credible for *source-only* FP16 and some *source+SASS* FP64 cases, while frontier LLM reasoning is most justified for *source+SASS* FP16 and the main FP32 triage cells.

5.4 RQ4: What Does the Signal Buy Under a Profiling Budget?

Answer. Ranking rows by predicted RAI relative to the balance point can move many true compute-bound kernels to the front of a fixed profiling queue, especially for FP16 with *source+SASS* and for FP32 even in *source-only* mode.

Table 4 makes the scarce-resource scenario explicit. If a tool profiles only the top 10% of kernel-device rows per precision, random ranking would recover 10% of the true compute-bound rows by construction. On the shared subset, *source+SASS* GPT-5.4 and Opus 4.6 recover all FP16 compute-bound rows at that budget, while their *source-only* variants recover none. FP32 is already strong *source-only*: the top 10% of GPT-5.4 rows recovers 82% of true compute-bound cases, and Opus 4.6 reaches 96%. FP64 remains harder, plateauing around 0.55–0.59 recall at the same budget.

The selected queues are also strongly enriched relative to random. At the same 10% budget, *source+SASS* FP16 queues for GPT-5.4 and Opus 4.6 are about 9.9× richer in true compute-bound rows than random, and *source-only* FP32 queues for GPT-5.4 and Opus 4.6 are about 8.1× and 9.5× richer. This is the main practical reading of the paper’s signal: it is useful not because it replaces profiling, but because it can prioritize which kernels reach the profiler first.

The 10% point is not the whole story. Appendix Table 9 shows that the ranking is broadly stable across 5/10/20/30% budgets, and it sharpens the comparison to the static baselines. For *source-only* FP16 queueing, the best learned static baseline is already stronger than any *source-only* LLM, reaching 0.471/0.765/0.971/1.000 recall across the sweep. For *source+SASS* FP16, the frontier LLM remains clearly better, with Opus 4.6 already at 0.971 recall at 5% budget and 1.000 by 10%. For *source-only* FP32, Opus 4.6 remains strongest at the front of the queue. For *source+SASS* FP64, a cheap SASS mnemonic ranker outperforms the LLMs across the whole sweep, again showing that some post-compilation regimes are largely recoverable from deterministic counts alone. Appendix Table 7 shows that these rankings are not merely high-recall because the models predict compute-bound everywhere. On FP32, *source-only* Opus 4.6 reaches compute-bound precision/recall/F1 of 1.000/0.880/0.936, and *source+SASS* raises that to 0.979/0.940/0.959. The FP16 failure mode is much sharper: every *source-only* model has zero compute-bound recall, while GPT OSS *source+SASS* reaches only 0.176 recall despite 0.750 precision. This is why we treat *source-only* FP16 triage as unusable and the frontier *source+SASS* FP16 lift as practically meaningful rather than cosmetic.

That practical value still has a price. The strongest models are also much more expensive, but the latency ranking is not monotonic.

Table 2 reports median per-query cost and latency on the shared subset. GPT OSS remains orders of magnitude cheaper than the frontier models: roughly \$0.0006 per *source-only* query and \$0.0008 with SASS, versus \$0.0197/\$0.0315 for GPT-5.4 and \$0.0679/\$0.1066 for Opus 4.6. Latency is different: GPT-5.4 is the fastest median response at roughly 11.3 s *source-only* and 12.3 s with SASS, while GPT OSS is the slowest at about 33 s in both settings.

Practical implication. The strongest deployment is a cascade, not an LLM-only stack. Cheap static rankers already cover *source-only* FP16 and *source+SASS* FP64 surprisingly well, while frontier LLMs justify their cost where *source-visible* semantics are insufficient, especially for *source+SASS* FP16 and the main FP32 triage regimes.

6 Discussion

What this enables for AI-assisted software engineering. The paper’s strongest positive result is not a raw error number; it is that off-the-shelf LLMs can often provide a *useful early decision proxy* from software artifacts alone. That is directly relevant to AI coding agents and developer-facing tools. Before spending target-GPU time, an agent can ask a narrower question than “will this be faster?”: is this kernel likely still on the memory side of the Roofline, or has

Table 5: Deployment routing summary from the shared subset. These are corpus-backed first-stage choices, not universal rules.

Regime	First stage	Why
source-only FP16	Static-first	One-shot LLMs stay at the trivial Task C baseline; cheap source features recover more queuing signal.
source-only FP32/FP64	Frontier LLM	Best pre-compilation triage overall, especially for FP32 where paired LLM wins are decisive.
source+SASS FP16/FP32	Frontier LLM	Largest paired gains once lowered arithmetic and DRAM-visible behavior become explicit.
source+SASS FP64	Static-first, LLM optional	Cheap SASS counts and learned SASS features already match or beat one-shot LLM triage in this regime.

the change plausibly shifted it toward the compute side? That is a simpler, earlier, and more defensible use of LLM-based performance reasoning. The budgeted retrieval result makes that point concrete: at a 10% profiling budget, *source+SASS* GPT-5.4 and Opus 4.6 recover all shared-subset FP16 compute-bound rows, and source-only FP32 rankings already recover 82–96%. The revised baseline comparison also makes the positive result narrower and more useful: the LLM is not the only viable execution-free triage mechanism, but it is the strongest option in the regimes where source-visible intent and post-compilation behavior diverge most.

Where source-only and source+SASS fit in a real workflow. The paper also clarifies that execution-free reasoning has two meaningful stages. *source-only* fits pre-compilation tasks such as code review, design exploration, and agent planning. *source+SASS* fits a later stage where the code can be compiled for a target architecture but the accelerator is still unavailable for execution or profiling. Treating both stages as the same “hardware-free” scenario obscures an important software-engineering distinction: compilation artifacts are themselves a development resource.

Why deterministic and learned static features still matter. The added reference baselines sharpen the methodological boundary. A simple lexical source rule is not enough to explain the useful FP32/FP64 source-only signal, and a runtime×GPU median baseline collapses outside the zero-heavy FP16 split. But supervised source-only baselines can still recover corpus-level triage cues, and architecture-specific SASS counts already expose substantial post-compilation structure after lowering. In a few regimes, especially source-only FP16 queueing and *source+SASS* FP64 triage, the static baselines are competitive with or stronger than one-shot prompting. That suggests a practical direction for tool builders: combine deterministic compiler or disassembly features, lightweight supervised models, and LLM reasoning in a cascade, rather than treating them as competing substitutes. Table 5 distills that cascade into the first-stage routing choice that the released evidence actually supports.

What the released rows look like in practice. Appendix Table 11 makes the aggregate story concrete with two released rows. In accuracy-cuda, the source-only explanation explicitly notes that compiler-generated HFMA2 instructions may exist but cannot be justified from source alone, so the model conservatively predicts zero FP16 work; once SASS is added, the same model attributes the nonzero FP16 signal to an executed HFMA2.MMA instruction

and matches the profiler almost exactly. In boxfilter-cuda, by contrast, the source already exposes the 21-tap loop trip count, the 64 output threads per block, and the roughly 1 MiB read footprint, so source-only FP32 reasoning is already effectively exact. That pair is the paper’s core deployment lesson in miniature: *source+SASS* matters most when lowering creates executed work that the source does not expose, not for every kernel.

What tool builders should and should not take from the results. Tool builders should use the method as a triage filter, not as a ground-truth oracle. The useful deployment is to prioritize profiling effort, annotate likely memory-side versus compute-side cases, or flag kernels whose source-level reasoning is too uncertain to trust. The unsafe deployment is to use the predictions as a runtime substitute, a speedup estimator, or a universal static analysis of GPU performance. The FP16 results make that limit especially clear: without post-compilation evidence, the models often miss work that only becomes visible after lowering.

What the exploratory failure map still suggests. We intentionally demote the feature/error heatmap to Appendix Figure 7 because its labels come from an auxiliary LLM-voting pipeline rather than human annotation. The appendix now adds a post-hoc multiple-comparison sanity check and raw kernel-prevalence counts for that figure, so the repeated positive cells are no longer just visual impressions. Even so, its descriptive pattern is consistent with the comparative results above: division, special math, reusable subexpressions, and loop-invariant floating-point work remain difficult largely because source-visible intent is a weak proxy for compiled arithmetic and DRAM-visible traffic in exactly those cases.

Why the benchmark release matters. Even with the method’s limitations, the released corpus is a useful research contribution in its own right. It exposes a benchmarkable software-engineering task with source files, build context, runtime arguments, SASS, profiler-derived labels, and model outputs. The added deterministic and learned reference baselines show why that matters: the benchmark is not just for ranking off-the-shelf LLMs, but for comparing multiple execution-free reasoning strategies that win in different regimes. It still supports several follow-on directions that the current paper cannot complete without new experiments: stronger compiler-grade analyzers, repeated-query robustness studies, and hybrid predictors that combine compiler features with LLM reasoning.

Why the claims are intentionally narrow. The measured evidence supports execution-free triage, not general GPU-performance prediction. That narrower claim is still valuable: it better matches the observed behavior, it better fits the needs of AI-assisted software engineering, and it gives future work a cleaner benchmark target.

7 Threats to Validity and Limitations

Benchmark representativeness. The corpus comes from HeCBench and covers only the 95 benchmark programs that yielded the released 254-kernel evaluation subset. That is a meaningful but still selective sample of CUDA and OpenMP GPU software. The results should therefore be read as evidence about this benchmarked task, not as a claim about all GPU kernels or all accelerator software.

Hardware scope. We study only NVIDIA GPUs, and only four of them: RTX 3080, A10, A100, and H100. The architectural generalization claims are therefore limited to those device families and their balance-point regimes. The benchmark does not cover AMD or Intel GPUs, CPU-only kernels, or other accelerator programming models such as HIP and SYCL.

Task scope. The prediction target is deliberately narrow. We predict only the first kernel invocation, only DRAM-level RAI, and only the per-precision FP16/FP32/FP64 decomposition used in this study. We do not predict runtime, throughput, speedup, cache-aware arithmetic intensity, or end-to-end application performance. Readers should therefore resist extrapolating the results beyond early first-invocation triage.

Ground-truth measurement. Ground truth comes from profiler counters, not from source-level reasoning. That is the right choice for this paper, but it means the denominator is DRAM-visible traffic rather than all source-visible loads and stores. Cache write-back behavior, replay effects, and profiler noise can therefore influence the target quantity. We reduce noise by averaging three first-invocation profiles, but this does not eliminate all measurement uncertainty.

Prompting and model stability. The released evaluation subset queries the expensive models once per completed kernel-device pair. We therefore cannot estimate run-to-run stochastic variability from the existing artifact, and closed-model behavior may drift across snapshots. We address this by narrowing claims and by reporting completed-row counts explicitly, but we cannot retroactively add repeated-query evidence.

Coverage imbalance. GPT OSS does not complete the full evaluation subset: it has 858 source-only and 845 *source*+SASS sample rows instead of 1,016. Cross-model comparisons are therefore restricted to the 732 kernel-device rows completed by all three models in both prompt settings. This is a fairer comparison, but it also means that the cheapest model’s reported quality is conditioned on the rows where it completed.

Baseline limitations. We now add context-only, deterministic, and lightweight learned tree baselines, and the learned rows remain stable across five seeds. But those baselines are still deliberately modest. The deterministic rows use approximate lexical and instruction-mix counts rather than compiler-grade analyzers. The learned rows are evaluated with grouped cross-validation on the released corpus rather than on a separately curated external training/test split. Stronger compiler-driven or larger-scale learned baselines could therefore do better than the references reported here.

Failure-analysis validity. The feature/error analysis is exploratory because its source-feature labels come from an auxiliary LLM-voting pipeline, not human annotation. We therefore treat the appendix heatmap and its new prevalence/BH-corrected sanity check only as descriptive evidence about recurring error patterns. It should not be interpreted as statistically validated causal evidence.

Possible benchmark leakage. Because the models are off-the-shelf and trained on large web corpora, some kernels or their close variants may have appeared in pretraining data. The paper cannot

Table 6: Ground-truth class balance on the 732-row shared subset used for all paired cross-model comparisons. The imbalance is precision-dependent: FP16/FP32 have more nonzero rows than FP64, but all three precisions remain zero-heavy, which is why Tasks A/C/D are reported separately.

Precision	Zero	Nonzero BB	Nonzero CB	Zero share
FP16	513	185	34	0.701
FP32	496	186	50	0.678
FP64	623	56	53	0.851

rule out such contamination completely. That is another reason to frame the result as an empirical benchmark study rather than a claim about intrinsic reasoning ability in isolation.

8 Conclusion

This paper reframes LLM-based GPU performance reasoning as an ICSE problem: execution-free early triage from software artifacts. Using source files, build context, runtime arguments, and optional SASS, off-the-shelf LLMs can often estimate a profiler-relevant RAI signal well enough to support early GPU-kernel triage. That statement is strongest for nonzero FP32/FP64 cases from source alone and for FP16 once post-compilation evidence is available. In a fixed 10% profiling-budget simulation, the strongest configurations move a large fraction of true compute-bound rows to the front of the queue. The added static baselines show a more nuanced landscape: simple deterministic and lightweight learned methods recover part of the same signal, and they are competitive in some source-only FP16 and *source*+SASS FP64 settings, but they do not eliminate the need for stronger reasoning on nonzero FP16/*source*+SASS FP32 triage or for accurate numeric RAI prediction.

The evidence does not support a stronger claim. These models do not replace profilers, they do not predict runtime in general, and they remain brittle when compiler-realized behavior diverges from surface syntax. What the paper does show is narrower and useful: execution-free RAI prediction is now a benchmarkable software-engineering task, and off-the-shelf prompting already provides a strong practical baseline within a broader space of static reasoning methods.

A Appendix

A.1 Additional Stratified Error Plots

A.2 Class Balance and Compute-Bound Precision/Recall

A.3 Static Reference Baselines

A.4 Budget Sweep and Seed Sensitivity

A.5 Illustrative Released Rows

A.6 Exploratory Failure Map

A.7 Anonymity Note

The current workspace contains public website and repository mirrors for the benchmark artifact. To preserve double anonymity, the paper text omits those URLs. The project README records the replacement points for a de-anonymized camera-ready version.

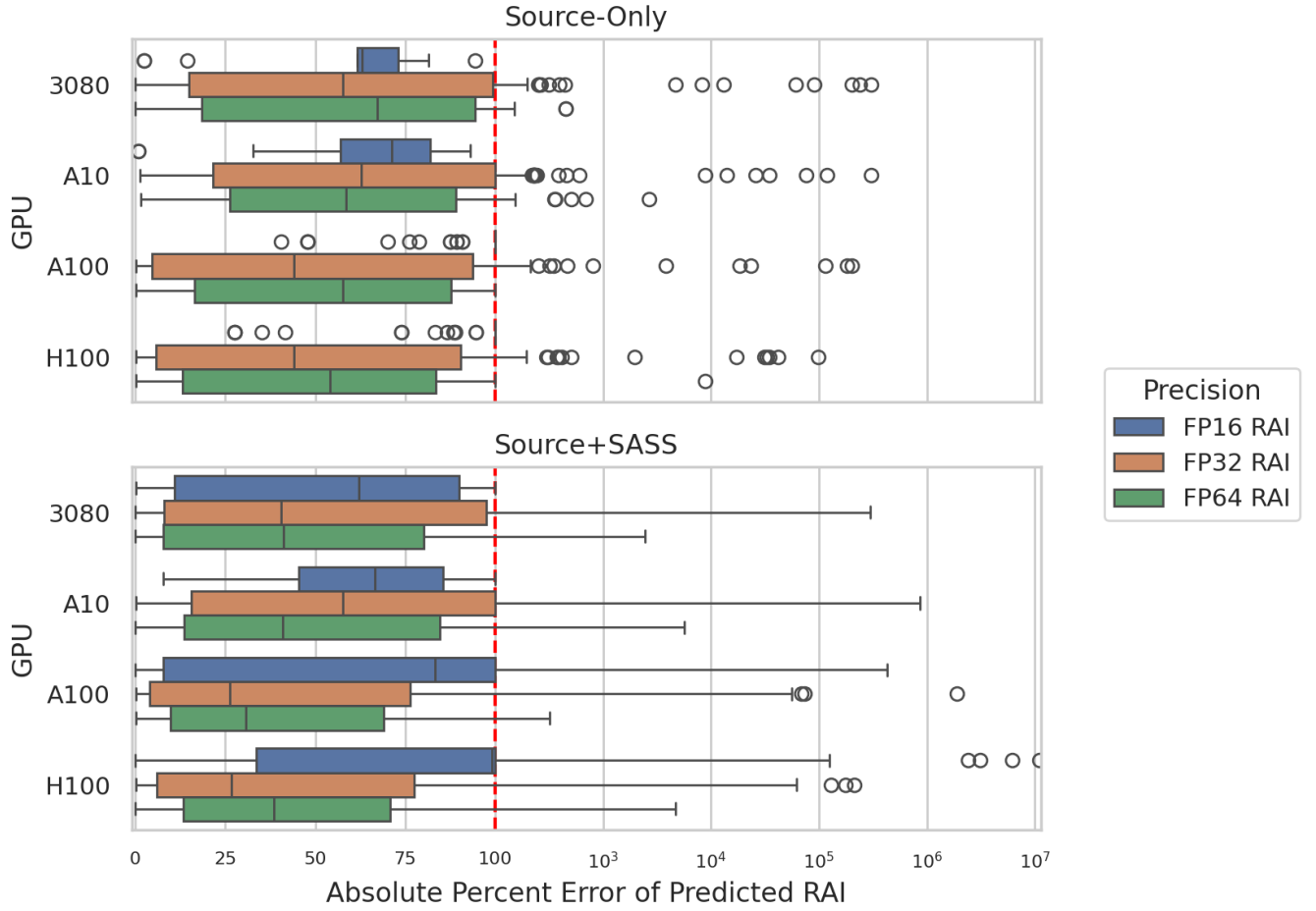


Figure 5: Absolute percent error on nonzero RAI cases, stratified by GPU. The smaller-cache 3080/A10 settings are generally less stable than A100/H100, especially for source-level reasoning about FP32/FP64 DRAM-visible traffic.

Table 7: Nonzero compute-bound precision/recall/F1 on the 732-row shared subset. These metrics complement Task C balanced accuracy by exposing whether a method reaches high recall by over-predicting the compute-bound side.

Model	Evidence	FP16 P/R/F1	FP32 P/R/F1	FP64 P/R/F1
GPT-5.4	source-only	- / 0.000 / -	0.909 / 0.800 / 0.851	0.950 / 0.358 / 0.521
GPT-5.4	source+SASS	0.730 / 0.794 / 0.761	0.952 / 0.800 / 0.870	0.903 / 0.528 / 0.667
GPT OSS	source-only	- / 0.000 / -	0.950 / 0.760 / 0.844	0.917 / 0.415 / 0.571
GPT OSS	source+SASS	0.750 / 0.176 / 0.286	0.949 / 0.740 / 0.831	0.900 / 0.340 / 0.493
Opus 4.6	source-only	- / 0.000 / -	1.000 / 0.880 / 0.936	0.966 / 0.528 / 0.683
Opus 4.6	source+SASS	0.850 / 1.000 / 0.919	0.979 / 0.940 / 0.959	0.938 / 0.566 / 0.706

References

- [1] Gargi Alavani, Kajal Varma, and Santonu Sarkar. 2018. Predicting Execution Time of CUDA Kernel Using Static Analysis. In *2018 IEEE Intl Conf on Parallel & Distributed Processing with Applications, Ubiquitous Computing & Communications, Big Data & Cloud Computing, Social Computing & Networking, Sustainable Computing & Communications (ISPA/IUCC/BDCloud/SocialCom/SustainCom)*. IEEE, Piscataway, NJ, USA, 948–955. doi:10.1109/BDCloud.2018.00139
- [2] Sara S. Baghsorkhi, Matthieu Delahaye, Sanjay J. Patel, William D. Gropp, and Wen-mei W. Hwu. 2010. An adaptive performance modeling tool for GPU architectures. In *Proceedings of the 15th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming (PPoPP '10)*. Association for Computing Machinery, New York, NY, USA, 105–114. doi:10.1145/1693453.1693470
- [3] Gregory Bolet, Giorgis Georgakoudis, Harshitha Menon, Konstantinos Parasyris, Niranjana Hasabnis, Hayden Estes, Kirk Cameron, and Gal Oren. 2025. Can Large Language Models Predict Parallel Code Performance?. In *Proceedings of the 34th International Symposium on High-Performance Parallel and Distributed Computing*. ACM, University of Notre Dame Conference Facilities Notre Dame IN USA, 1–6. doi:10.1145/3731545.3743645
- [4] Lorenz Braun, Sotirios Nikas, Chen Song, Vincent Heuveline, and Holger Fröning. 2021. A Simple Model for Portable and Fast Prediction of Execution Time and Power Consumption of GPU Kernels. *ACM Trans. Archit. Code Optim.* 18, 1 (Dec. 2021), 7:1–7:25. doi:10.1145/3431731
- [5] Wentao Chen, Jiace Zhu, Qi Fan, Yehan Ma, and An Zou. 2025. CUDA-LLM: LLMs Can Write Efficient CUDA Kernels. arXiv:2506.09092 [cs]. doi:10.48550/arXiv.2506.09092

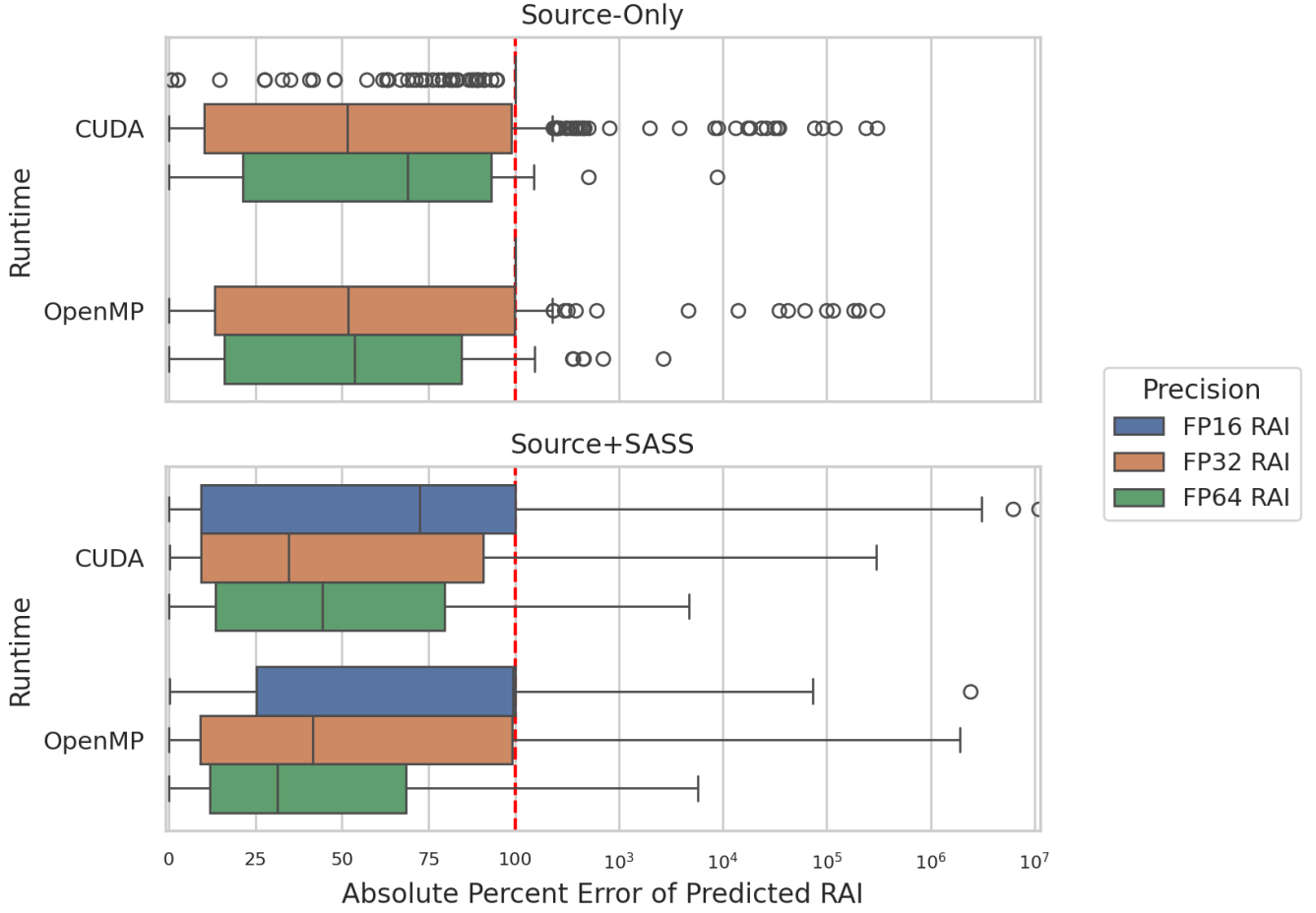


Figure 6: Absolute percent error on nonzero RAI cases, stratified by runtime. The runtime split is precision-sensitive rather than uniform: CUDA tends to be easier for FP16/FP32 triage, while FP64 remains mixed across CUDA and OpenMP.

Table 8: Static reference baselines on the 732-row shared subset. Context median is a grouped cross-validated runtime×GPU baseline; lexical and mnemonic are deterministic heuristics; the learned rows are five-fold grouped tree baselines grouped by program. For each precision, the slash-separated triple reports Task A zero/nonzero balanced accuracy, Task C nonzero BB/CB balanced accuracy, and Task D three-way balanced accuracy. “MedAPE” is the median nonzero absolute percent error.

Baseline	FP16 A/C/D	FP32 A/C/D	FP64 A/C/D	FP16 MedAPE	FP32 MedAPE	FP64 MedAPE
Context median CV	0.790 / 0.500 / 0.535	0.500 / 0.500 / 0.333	0.500 / 0.500 / 0.333	100.0	100.0	100.0
Source lexical	0.523 / 0.500 / 0.351	0.693 / 0.489 / 0.444	0.651 / 0.683 / 0.531	100.0	99.7	99.5
SASS mnemonic	0.983 / 0.500 / 0.656	0.897 / 0.577 / 0.649	0.995 / 0.756 / 0.834	95.9	64.9	68.8
Source learned RF	0.856 / 0.688 / 0.691	0.829 / 0.815 / 0.744	0.719 / 0.663 / 0.608	141.6	100.0	100.0
Source learned ET	0.856 / 0.699 / 0.697	0.796 / 0.773 / 0.688	0.695 / 0.583 / 0.515	217.4	100.0	100.0
SASS learned RF	0.970 / 0.567 / 0.690	0.913 / 0.737 / 0.754	0.899 / 0.824 / 0.817	161.2	100.0	67.1
SASS learned ET	0.909 / 0.591 / 0.677	0.838 / 0.739 / 0.696	0.862 / 0.744 / 0.722	178.6	100.0	100.0

- [6] Yannick Emonds, Lorenz Braun, and Holger Fröning. 2023. CUDAsap: Statically-Determined Execution Statistics as Alternative to Execution-Based Profiling. In *2023 IEEE/ACM 23rd International Symposium on Cluster, Cloud and Internet Computing (CCGrid)*. IEEE, Piscataway, NJ, USA, 119–130. doi:10.1109/CCGrid57682.2023.00021
- [7] Jingzhi Gong, Vardan Voskanyan, Paul Brookes, Fan Wu, Wei Jie, Jie Xu, Rafail Giavrimis, Mike Basios, Leslie Kanthan, and Zheng Wang. 2025. Language Models for Code Optimization: Survey, Challenges and Future Directions. arXiv:2501.01277 [cs]. doi:10.48550/arXiv.2501.01277

- [8] João Guerreiro, Aleksandar Ilic, Nuno Roma, and Pedro Tomás. 2019. GPU Static Modeling Using PTX and Deep Structured Learning. *IEEE Access* 7 (2019), 159150–159161. doi:10.1109/ACCESS.2019.2951218
- [9] Sunpyo Hong and Hyesoon Kim. 2009. An analytical model for a GPU architecture with memory-level and thread-level parallelism awareness. *SIGARCH Comput. Archit. News* 37, 3 (June 2009), 152–163. doi:10.1145/1555815.1555775
- [10] Naman Jain, King Han, Alex Gu, Wen-Ding Li, Fanjia Yan, Tianjun Zhang, Sida Wang, Armando Solar-Lezama, Koushik Sen, and Ion Stoica. 2024. Live-CodeBench: Holistic and Contamination Free Evaluation of Large Language Models for Code. arXiv:2403.07974 [cs]. doi:10.48550/arXiv.2403.07974

Table 9: Budget-sweep compute-bound recall for the strongest source-only and source+SASS methods in each family, selected by mean recall across the 5/10/20/30% sweep on the shared subset. This shows whether the ranking behind Table 4 is stable beyond the single 10% operating point.

Precision	Method	R@5%	R@10%	R@20%	R@30%
FP16	source-only best LLM (Opus 4.6)	0.000	0.000	0.324	0.324
FP16	source-only best static (Source learned ET)	0.471	0.765	0.971	1.000
FP16	source+SASS best LLM (Opus 4.6)	0.971	1.000	1.000	1.000
FP16	source+SASS best static (SASS mnemonic)	0.294	0.706	0.971	1.000
FP32	source-only best LLM (Opus 4.6)	0.740	0.960	0.960	0.960
FP32	source-only best static (Source learned ET)	0.480	0.700	0.960	0.960
FP32	source+SASS best LLM (Opus 4.6)	0.660	0.940	0.940	0.940
FP32	source+SASS best static (SASS mnemonic)	0.440	0.760	1.000	1.000
FP64	source-only best LLM (Opus 4.6)	0.208	0.585	0.604	0.604
FP64	source-only best static (Source learned ET)	0.302	0.453	0.453	0.453
FP64	source+SASS best LLM (Opus 4.6)	0.208	0.566	0.566	0.566
FP64	source+SASS best static (SASS mnemonic)	0.604	0.811	1.000	1.000

Table 10: Seed sensitivity for the learned static baselines across five grouped-CV seeds. Each entry reports mean±std over seeds for Task C balanced accuracy and nonzero MedAPE. The low spread shows that the static-baseline comparisons are not driven by an unusually favorable random seed.

Baseline	FP16 C / MedAPE	FP32 C / MedAPE	FP64 C / MedAPE	FP16 std(C)	FP32 std(C)	FP64 std(C)
Source learned RF	0.710 / 115.2	0.801 / 100.0	0.661 / 100.0	0.028	0.010	0.009
Source learned ET	0.674 / 227.2	0.767 / 100.0	0.585 / 100.0	0.034	0.025	0.019
SASS learned RF	0.578 / 182.3	0.736 / 100.0	0.831 / 73.6	0.014	0.001	0.007
SASS learned ET	0.591 / 174.5	0.735 / 100.0	0.720 / 100.0	0.011	0.004	0.044

Table 11: Illustrative released rows used only for intuition, not as additional evaluation evidence. These examples show the two clearest deployment regimes in the released artifact: hidden lowered FP16 work that requires post-compilation evidence, and source-visible FP32 work that source-only reasoning already captures well.

Case	Released row	Expected → predicted RAI	Why it matters
Hidden lowered FP16 work	accuracy-cuda, A100, Opus 4.6	FP16 source-only: 0.0254 → 0.0; FP16 source+SASS: 0.0254 → 0.0256	The source-only explanation notes that compiler-generated HFMA2 materialization may exist but cannot be justified from source alone, so it conservatively predicts zero FP16 work. The SASS-backed explanation instead points to an executed HFMA2.MMA instruction on the taken path and counts it as real FP16 arithmetic.
Source-visible FP32 work	boxfilter-cuda, A100, Opus 4.6	FP32 source-only: 21.975 → 22.0	The source already exposes the 21-tap loop trip count, the 64 output threads per block, and the roughly 1 MiB unique read footprint. In this regime, source-only reasoning is already enough and post-compilation evidence adds little.

- [11] Zheming Jin and Jeffrey S. Vetter. 2023. A Benchmark Suite for Improving Performance Portability of the SYCL Programming Model. In *2023 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*. IEEE, Piscataway, NJ, USA, 325–327. doi:10.1109/ISPASS57527.2023.00041
- [12] Tal Kadosh, Niranjana Hasabnis, Timothy Mattson, Yuval Pinter, and Gal Oren. 2023. Quantifying OpenMP: Statistical Insights into Usage and Adoption. In *2023 IEEE High Performance Extreme Computing Conference (HPEC)*. IEEE, Piscataway, NJ, USA, 1–7. ISSN: 2643-1971. doi:10.1109/HPEC58863.2023.10363459
- [13] Erel Kaplan, Tomer Bitan, Lian Ghayeb, Le Chen, Tom Yotam, Niranjana Hasabnis, and Gal Oren. 2026. ParaCodex: A Profiling-Guided Autonomous Coding Agent for Reliable Parallel Code Generation and Translation. arXiv:2601.04327 [cs]. doi:10.48550/arXiv.2601.04327
- [14] Hao Li, Haoxiang Zhang, and Ahmed E. Hassan. 2025. The Rise of AI Teammates in Software Engineering (SE) 3.0: How Autonomous Coding Agents Are Reshaping Software Engineering. arXiv:2507.15003 [cs]. doi:10.48550/arXiv.2507.15003
- [15] Chenxiao Liu, Shuai Lu, Weizhu Chen, Daxin Jiang, Alexey Svyatkovskiy, Shengyu Fu, Neel Sundaresan, and Nan Duan. 2023. Code Execution with Pre-trained Language Models. arXiv:2305.05383 [cs]. doi:10.48550/arXiv.2305.05383
- [16] Chenyang Lyu, Lecheng Yan, Rui Xing, Wenxi Li, Younes Samih, Tianbo Ji, and Longyue Wang. 2024. Large Language Models as Code Executors: An Exploratory Study. arXiv:2410.06667 [cs]. doi:10.48550/arXiv.2410.06667
- [17] Minh-Khoi Nguyen-Nhat, Hoang Duy Nguyen Do, Huyen Thao Le, and Thanh Tuan Dao. 2024. LLMPerf: GPU Performance Modeling meets Large Language Models. In *2024 32nd International Conference on Modeling, Analysis and Simulation of Computer and Telecommunication Systems (MASCOTS)*. IEEE, Piscataway, NJ, USA, 1–8. doi:10.1109/MASCOTS64422.2024.10786558
- [18] Anne Ouyang, Simon Guo, Simran Arora, Alex L. Zhang, William Hu, Christopher Ré, and Azalia Mirhoseini. 2025. KernelBench: Can LLMs Write Efficient GPU Kernels? arXiv:2502.10517 [cs]. doi:10.48550/arXiv.2502.10517
- [19] Yun Peng, Akhilesh Deepak Gotmare, Michael R. Lyu, Caiming Xiong, Silvio Savarese, and Doyen Sahoo. 2025. PerfCodeGen: Improving Performance of LLM Generated Code with Execution Feedback. In *2025 IEEE/ACM Second International Conference on AI Foundation Models and Software Engineering (Forge)*. IEEE, Piscataway, NJ, USA, 1–13. doi:10.1109/Forge66646.2025.00008
- [20] Srivishnu Pyda, Daniel Nichols, and Abhinav Bhatle. 2025. The Shortcomings of Code LLMs in Modeling Code Properties. In *2025 IEEE/ACM International Workshop on Large Language Models for Code (LLM4Code)*. IEEE, Piscataway, NJ, USA, 193–199. doi:10.1109/LLM4Code66737.2025.00029
- [21] Hong-Quan Tran, Van-Sang Pham, Tien-Son Pham, Tuan-Duc Chu, Anh-Tuan Mai, Xuan Truong Nguyen, and Thanh-Tuan Dao. 2025. Accurate Latency Predictor for Triton-Based GPU Kernels with PTX Features and XGBoost. In *2025 RIVF International Conference on Computing and Communication Technologies (RIVF)*. IEEE, Piscataway, NJ, USA, 980–985. ISSN: 2473-0130. doi:10.1109/RIVF68649.2025.11365185
- [22] Zixian Wang, Cole Ramos, Muhammad A. Awad, and Keith Lowery. 2025. Omniverse: Predicting GPU Kernels Performance with LLMs. arXiv:2506.20886 [cs]. doi:10.48550/arXiv.2506.20886
- [23] Samuel Williams, Andrew Waterman, and David A. Patterson. 2009. Roofline: an insightful visual performance model for multicore architectures. *Communications of The ACM* 52, 4 (April 2009), 65–76. MAG ID: 200255321. doi:10.1145/1498765.1498785

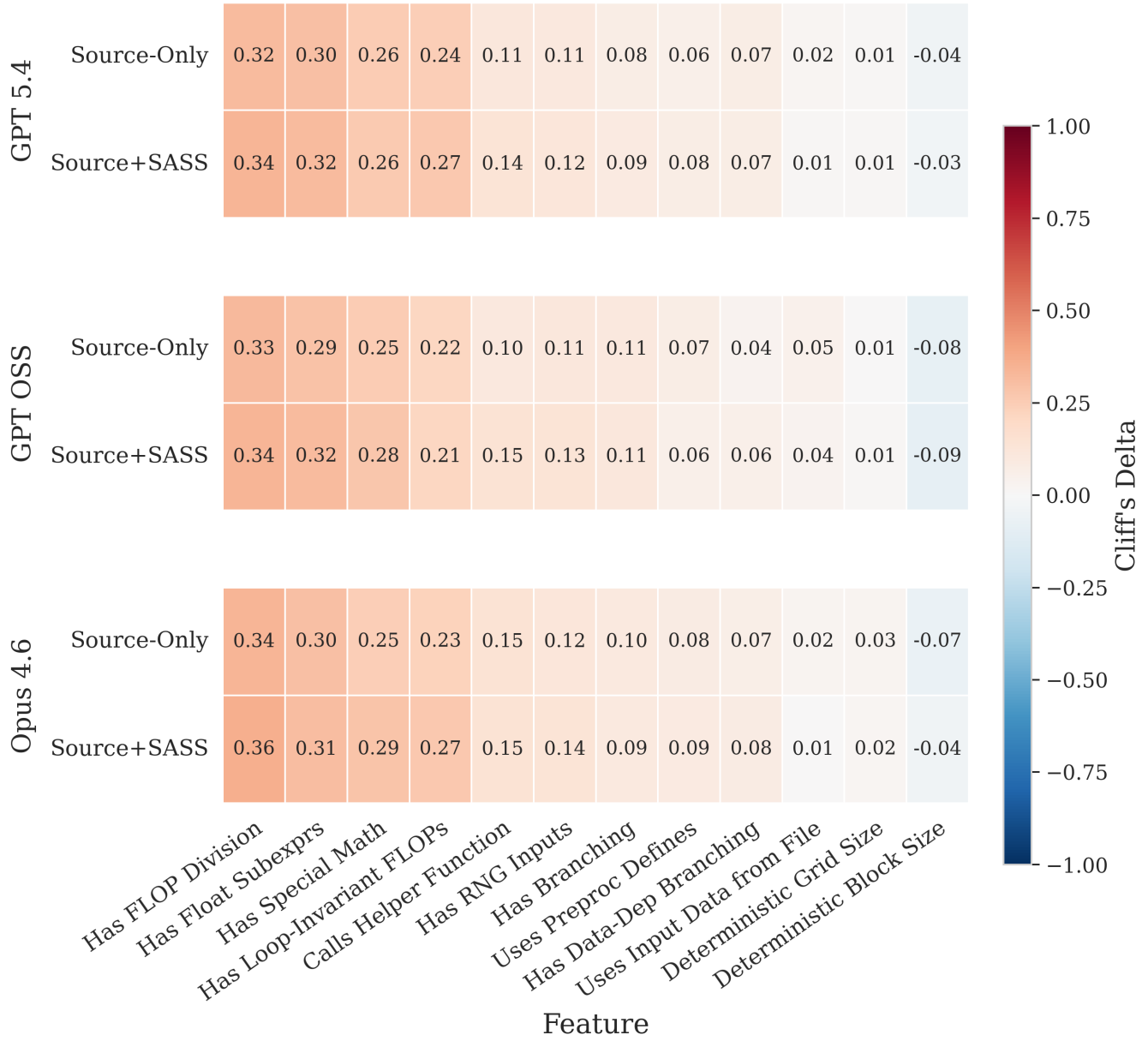


Figure 7: Exploratory feature/error association heatmap from the released artifact. Positive Cliff’s delta means higher absolute RAI error when the feature is present. The strongest repeated error signals come from division, special math, reusable subexpressions, and loop-invariant floating-point work. A post-hoc Mann–Whitney/BH sanity check across the same 72 model×prompt×feature cells preserves repeated positive associations for division, common float subexpressions, special math, loop-invariant FLOPs, helper-function calls, RNG inputs, branching, preprocessor defines, and data-dependent branching; those features appear in 7–90% of the 1,336 voted kernels. Because the feature labels are generated by an auxiliary LLM-voting pipeline, we treat this figure as descriptive rather than causal.

[24] Danning Xie, Mingwei Zheng, Xuwei Liu, Jiannan Wang, Chengpeng Wang, Lin Tan, and Xiangyu Zhang. 2025. CORE: Benchmarking LLMs Code Reasoning Capabilities through Static Analysis Tasks. arXiv:2507.05269 [cs]. doi:10.48550/arXiv:2507.05269

[25] Zijian Zhang, Rong Wang, Shiyang Li, Yuebo Luo, Mingyi Hong, and Caiwen Ding. 2025. CudaForge: An Agent Framework with Hardware Feedback for CUDA